

# Question 1

Your task is to implement a PHP function using both an **arrow function** and a **callback function**.

## Instructions:

1. Create a PHP function named `processNumbers` that accepts an array and a callback function.
2. Use an **arrow function** as your callback when testing the function.
3. Print the resulting array after applying the callback.

```
Call processNumbers([1,2,3,4], fn($n) => $n * 2)
Expected result: [2,4,6,8]
```

# Question 2

Your task is to create a PHP function that demonstrates the use of multiple array functions:

- `count`
- `array_reverse`
- `array_push`
- `array_pop`
- `array_unshift`
- `array_shift`
- `array_map`
- `array_filter`
- `array_reduce`

**Function Name:** `arrayOperationsPractice`

**Purpose:** This function takes an array of numbers, performs multiple operations using all the above functions, and prints a structured summary of results.

## Instructions:

1. Create a PHP function named `arrayOperationsPractice` that accepts an array of numbers.
2. Inside the function, perform the following operations in order:
  - **Count:** Use `count` to get the total number of elements.
  - **Reverse:** Use `array_reverse` to reverse the array.
  - **Push:** Use `array_push` to add the number `10` at the end.

- **Pop:** Use `array_pop` to remove the last element and store it.
- **Unshift:** Use `array_unshift` to add the number `0` at the beginning.
- **Shift:** Use `array_shift` to remove the first element and store it.
- **Map:** Use `array_map` to multiply each element by `2`.
- **Filter:** Use `array_filter` to keep only even numbers.
- **Reduce:** Use `array_reduce` to calculate the sum of all remaining elements.

3. Return or print an associative array summarizing each step so the output is clear.

Example call:

```
arrayOperationsPractice([1, 2, 3, 4, 5])
```

Expected printed output (array keys show operation names):

```
Array
(
    [count] => 5
    [reversed] => Array ( [0] => 5 [1] => 4 [2] => 3 [3] => 2 [4] => 1 )
    [after_push] => Array ( [0] => 5 [1] => 4 [2] => 3 [3] => 2 [4] => 1 [5] => 10 )
    [popped_value] => 10
    [after_unshift] => Array ( [0] => 0 [1] => 5 [2] => 4 [3] => 3 [4] => 2 [5] => 1 )
    [shifted_value] => 0
    [after_map] => Array ( [0] => 10 [1] => 8 [2] => 6 [3] => 2 )
    [after_filter] => Array ( [0] => 10 [1] => 8 [2] => 6 [3] => 2 )
    [sum] => 26
)
```

Notes:

- **Map:** multiply each element by 2.
- **Filter:** keep only even numbers.
- **Reduce:** calculate the sum of all elements after filter.